Day 91: Causal Language Modeling — Next-Token Prediction From Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 1 | Complete study guide

Watch: Building makemore Part 2: MLP (Andrej Karpathy) | Language Model Evaluation and Perplexity (Machine Learning TV)

Script: `day91_causal_language_modeling.py` · every number in this guide is copied by program from `day91_run_output.txt`, the real output of that script.

A language model is a chain of next-token predictions, and that is only a real probability model if no prediction can see its own answer: summed over every string, a causal model gives 1.000000; the same weights without the causal mask give 1.051473. Then three ways to make the predictions, on Day 90's Shakespeare split. The surprise is the counting model: with Kneser-Ney smoothing it reaches 2.197 bits per character, far better than Day 90's GPT (2.525) and than a fixed-window neural model (2.633). That is the number the rest of Week 14 has to beat.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

A **causal language model** predicts each token from the tokens before it, never after. By the **chain rule** of probability, the probability of a whole text is the product of these next-token probabilities, so training on next-token prediction is training a model of text. The loss is the mean of –log p(next token); divided by ln 2 it is **bits per token**, and 2^{bits} is the **perplexity**.

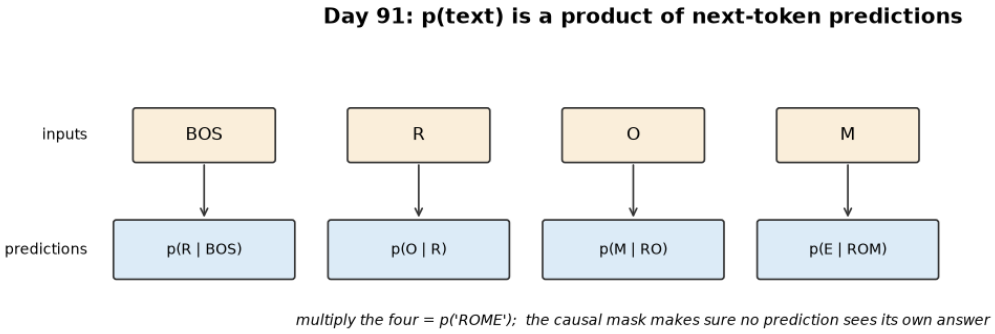
REAL-WORLD SCENARIO

Reading a sentence aloud one word at a time while a friend guesses the next word; the fewer surprises, the better the friend knows the language.

MATHEMATICS

$$p(x_1, \ldots, x_n) = \Pi_t\, p(x_t \mid x_1, \ldots, x_{t-1}); \quad \text{bits per token} = -(1/n)\, \Sigma_t \log_2 p(x_t \mid x_{<t}); \quad \text{perplexity} = 2^{\text{bits}}$$

SYSTEM DIAGRAM



A text's probability as a product of next-token predictions.

Today's part for Day 97

`day91_exercise.py` is the Kneser-Ney character model in one class, checked so that every context's probabilities add up to 1. Day 97's build uses it as the counting baseline.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The chain rule, checked

DEFINITION — start here, no prior knowledge assumed

A model is a probability model of text if the probabilities it gives to all possible texts add up to 1. With three tokens and length 4 there are 81 strings: few enough to list them all and add.

REAL-WORLD SCENARIO

Checking that a die's six faces really add up to certainty before betting on it.

SPECIFICATION

Model	Day 90's GPT, untrained, d = 8, 2 blocks, 3 tokens plus a beginning-of-sequence token, weights moved off their start.
Without the mask	the same weights run through Day 78's encoder block, where every position sees every other.

Why this test

The causal mask is usually explained as 'not cheating'. This shows what cheating breaks: the numbers stop being probabilities.

IMPLEMENTATION

```
def sequence_logprob(P, seqs, causal, h=2):
    BOS = P["E"].shape[0] - 1
    x = np.concatenate([np.full((len(seqs), 1), BOS), seqs[:, :-1]], axis=1)
    n = x.shape[1]
    if causal:
        lg, _ = d90.gpt_forward(x, P, h)
    else:
        z = d90.embed(P["E"], x) + d90.embed(P["pos"], np.arange(n))
        for k in sorted(P["blocks"]):
            z, _ = d90.w12.encoder_block(z, P["blocks"][k], h, np.ones(x.shape, bool))
        lg = d90.output_layer(z, P["Wout"], P["bout"])
    logp = d90.log_softmax(lg[..., :BOS])
    return np.take_along_axis(logp, seqs[..., None], axis=-1)[..., 0].sum(axis=1)
```

ACTUAL OUTPUT

```
causal (Day 90)              : sum over 81 strings = 1.000000; largest 0.8381, smallest 2.52e-10
no causal mask (Day 78 block) : sum over 81 strings = 1.051473; largest 0.8528, smallest 2.94e-10
Actual output from this lesson's run.
```

What it means

With the mask the 81 probabilities add up to 1.000000, whatever the weights. Without it they add up to 1.051473: each 'next-token probability' could look at the token it was supposed to predict, so their products no longer describe any distribution over texts. A non-causal model can still be useful (Day 78's encoder), but not for scoring or writing text left to right.

WHY CHOOSE THIS?

A proof by enumeration: no approximation anywhere.

WHY NOT THIS? (TRADEOFFS)

Only possible for tiny vocabularies and lengths.

2. Counting models: add-k against Kneser-Ney

DEFINITION — start here, no prior knowledge assumed

Both predict the next character from counts of what followed the same context in the training text. Add-k (Day 90) adds k to every count. **Kneser-Ney** subtracts a discount D from every seen count, gives the saved probability to the next shorter context, and below the top order uses **continuation counts**: how many different characters came before a string, not how often it occurred.

REAL-WORLD SCENARIO

A tourist guesses the next word of a phrase book; a local also knows which words fit after almost anything.

MATHEMATICS

$$p_{KN}(c \mid h) = max(count(h\,c) - D, 0) / count(h) + D \cdot N_{1+}(h\,\bullet) / count(h) \cdot p_{KN}(c \mid h')$$

h' = h without its first character; $N_{1+}(h\,\bullet)$ *= how many different characters followed h; lower orders count distinct left neighbours instead of occurrences*

IMPLEMENTATION

```
def prob(self, ctx, ch, k=None):
    k = self.n if k is None else k
    if k == 0:
        return 1.0 / len(self.V)
    src, tot, kinds = self.tab[k]
    h = ctx[len(ctx) - (k - 1):] if k > 1 else ""
    if tot[h] == 0:
        return self.prob(ctx, ch, k - 1)
    top = max(src[h + ch] - self.D, 0.0) / tot[h]
    lam = self.D * kinds[h] / tot[h]
    return top + lam * self.prob(ctx, ch, k - 1)
```

ACTUAL OUTPUT

```
discount chosen on held-back training text: D = 0.9 (tried 0.5, 0.75, 0.9, 0.95, 0.99)
  context    add-k  Kneser-Ney  contexts seen
        0    4.829       4.829              1
        1    3.583       3.583             65
        2    2.952       2.944          1,403
        3    2.574       2.485         11,556
        4    2.554       2.257         50,712
        5    2.895       2.205        141,021
        6        -       2.197        283,312
        7        -       2.201        447,349
        8        -       2.209        609,653
(43 s for all the counting models)
Actual output from this lesson's run.
```

What it means

With 0 or 1 character of context both are the same (4.829, 3.583). With more context add-k stalls and then gets worse (2.554 at 4, 2.895 at 5), because most long contexts are seen once and k spreads their probability too evenly. Kneser-Ney keeps improving to 2.197 at 6 characters and only then flattens (2.209 at 8). D = 0.9 was chosen on held-back training text, inside the tried range. It built and scored nine models on a million characters in under a minute.

WHY CHOOSE THIS?

Instant, no training, and hard to beat at short range.

WHY NOT THIS? (TRADEOFFS)

Cannot use context it has not seen verbatim: past about 6 characters, no gain.

3. A fixed-window neural model

DEFINITION — start here, no prior knowledge assumed

Bengio et al. (2003): the last k characters, each turned into a learned vector by one shared table, side by side into a tanh layer, then a softmax over the 65 characters. Unlike counting, similar contexts share statistics through the vectors.

REAL-WORLD SCENARIO

Guessing the next letter from the last few, having learned which letters behave alike.

SPECIFICATION

Model	24 numbers per character, 256 tanh units, output layer × 0.1.
Training	Adam (Day 37, clipping Day 60), learning rate 0.003, 4,000 steps of 512 random positions; weights seed 0, batches seed 1.
Measured	the same held-out characters as Section 2.

IMPLEMENTATION

```
class WindowLM:
    def __init__(self, V, k, seed=0):
        rng = np.random.RandomState(seed)
        self.k = k
        self.P = dict(E=rng.randn(V, EMB) * 0.1, W1=rng.randn(k * EMB, HID) / np.sqrt(k * EMB),
            b1=np.zeros(HID),
                      W2=rng.randn(HID, V) / np.sqrt(HID) * 0.1, b2=np.zeros(V))
    def logits(self, X):
        self.X = X; self.e = self.P["E"][X].reshape(len(X), -1)
        self.hd = np.tanh(self.e @ self.P["W1"] + self.P["b1"])
        return self.hd @ self.P["W2"] + self.P["b2"]
    def grads(self, X, y):
        lg = self.logits(X); n = len(y)
        loss, dz = d90.softmax_xent(lg, y)
        G = dict(W2=self.hd.T @ dz, b2=dz.sum(axis=0))
        dpre = (dz @ self.P["W2"].T) * (1 - self.hd ** 2)
        G["W1"], G["b1"] = self.e.T @ dpre, dpre.sum(axis=0)
        G["E"] = d90.embed_backward((dpre @ self.P["W1"].T).reshape(n, self.k, EMB), self.P["E"],
            X)
        return loss, G
```

ACTUAL OUTPUT

```
   k  weights  bits/char  seconds
   2    30809      3.043       49
   4    43097      2.653       51
   8    67673      2.633       71
  16   116825      2.685      100
Actual output from this lesson's run.
```

What it means

More context helps up to k = 8 (2.633), then k = 16 is worse (2.685) with nearly twice the weights: 4,000 steps of one prediction per window is probably too little training for the larger input; that was not tested further. Even the best window model is well behind Kneser-Ney at the same context length. Each window gives one prediction per forward pass, where a GPT gives one per position.

WHY CHOOSE THIS?

Shares statistics between similar contexts; small.

WHY NOT THIS? (TRADEOFFS)

Fixed window; one prediction per window makes training slow.

4. Side by side

DEFINITION — start here, no prior knowledge assumed

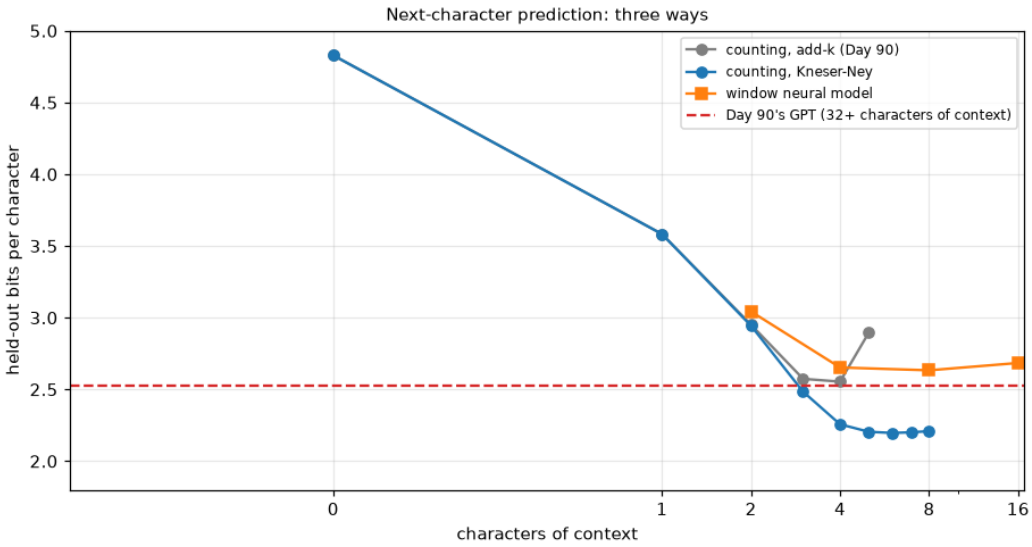
The best of each family, and Day 90's GPT as measured on Day 90.

REAL-WORLD SCENARIO

A league table before the season starts.

ACTUAL OUTPUT

```
Kneser-Ney, best (6 characters of context): 2.197 bits per character
window model, best (k = 8):                 2.633
Day 90's GPT, at least 32 characters of context: 2.525 (Day 90's run output; scored on Day 90's
    windows)
Day 90's add-k, 4 characters of context:         2.554 (Day 90's run output)
Actual output from this lesson's run.
```



Held-out bits per character against characters of context; the dashed line is Day 90's GPT.

What it means

On this data the counting model is the one to beat: 2.197, against 2.525 for Day 90's GPT (scored on Day 90's windows, so not exactly the same characters, but close in kind). A Transformer's strength is using long context; Day 90's was small and briefly trained, and whether more size and training let it use that context is Day 92's question.

WHY CHOOSE THIS?

An honest baseline, before any neural model is called good.

WHY NOT THIS? (TRADEOFFS)

Different models scored on slightly different positions are close, not identical, comparisons.

Interview preparation — technical questions

Q: How does the chain rule turn text modeling into next-token prediction?

A: p(text) = product of p(token | tokens before); its log is a sum of per-token log-probabilities.

Q: What is perplexity?

A: 2 to the power of the bits per token: the effective number of equally likely choices.

Q: What does Kneser-Ney do differently from add-k?

A: Discounts seen counts, interpolates with shorter contexts, and uses continuation counts below the top order.

Q: Why a beginning-of-sequence token?

A: So the first token also has a context and its probability is defined.

Q: Why can't a model without a causal mask score text?

A: Its predictions see the answers; here its 'probabilities' summed to 1.051.

Interview preparation — theoretical questions

Q: Why does add-k get worse with more context?

A: Long contexts are mostly seen once; adding k gives unseen characters far too much probability there.

Q: Why do continuation counts help?

A: A character's chance to follow a new context depends on how many contexts it follows, not how often.

Q: Why train a GPT if counting does this well?

A: Counting stops improving past about 6 characters; a Transformer can use far longer context, if trained well enough.

Q: Why is a fixed-window model slow to train?

A: One prediction per forward pass, where a causal Transformer predicts every position at once.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Next-token prediction with a causal mask defines a real probability for every text (the 81 probabilities sum to 1 to six decimals; without the mask, 1.051). On a million characters of Shakespeare, a well-smoothed counting model (Kneser-Ney, 2.197 bits per character) beats both a fixed-window neural model (2.633) and Day 90's GPT (2.525).

- Chain rule: 1.000000 causal, 1.051473 without the mask.
- Kneser-Ney: 2.197 bits per character at 6 characters of context; add-k at best 2.554.
- Window neural model: 2.633 at k = 8; k = 16 worse (2.685).
- The target for Day 92: below 2.197.

Suggested watch order, if starting from zero

- 1. Language Model Evaluation and Perplexity (Machine Learning TV) — what perplexity measures.
- 2. Building makemore Part 2: MLP (Andrej Karpathy) — Bengio's window model, built step by step.

Day 92 preview

- **Training the mini-GPT properly.** A learning-rate schedule, a short real sweep for the peak rate, a bigger model, full-context measurement and perplexity with its unit: does it beat 2.197?